

幕墙震动预警系统源代码阅读报告

项目概述

本项目是一个基于Nuxt.js的**幕墙震动数据检测与展示系统**，主要用于监测建筑幕墙的振动数据，实现实时预警和异常检测功能。系统支持多种传感器类型（加速度计和应变计），提供多维度的数据可视化和参数配置。

系统架构

技术栈

- **前端框架:** Vue 3 + Nuxt.js + TypeScript
- **UI组件库:** Element Plus
- **图表库:** ECharts
- **HTTP客户端:** Axios
- **样式:** Tailwind CSS + 自定义样式

项目结构

```
pages/vibration/
├── index.vue           # 主页面 - 数据检测与展示
├── dashboard.vue      # 仪表盘 - 实时监控
├── abnormal.vue       # 异常监控 - 数据统计与查询
└── parameter.vue      # 参数设置 - 设备配置与阈值

components/            # 通用组件
├── DatePicker.vue
├── DateRangePicker.vue
└── ...

layouts/
└── default.vue        # 默认布局（包含导航菜单）

store/                 # 状态管理
├── index.js
└── user.js
```

核心功能模块

1. 振动数据检测与展示 (pages/vibration/index.vue)

概述与职责: 此页面是振动数据的“探索与快速查询”入口，面向需要切换设备、选择数据粒度并快速查看图表的用。主要负责：

- 提供设备/类型/级别的选择控件
- 根据选择请求后端数据（秒/分/时/天/月）
- 组织并预处理数据供 ECharts 绘图使用

- 提供“切换标签页”以查看单轴或多轴数据

模板结构:

- 顶部工具栏 (UDashboardToolbar) : 选择设备、数据级别、确认按钮
- 标签页导航 (accelerometerTabs / strainGaugeTabs) : 切换显示 X/Y/Z 或 Ch1/Ch2
- 图表容器: #chart-x, #chart-y, #chart-z, #chart-all, #chart-ch1, #chart-ch2, #chart-strain-all

主要响应式状态:

- `selectedCategory`, `selectedDevice`, `selectedDeviceType`, `selectedDataSource`: 控制选择
- `accumulatedData`: 最终传给图表的数据结构, 包含 x/y/z/ch1/ch2 的 times & values
- `charts`、`chartOptions`: 缓存 ECharts 实例与预生成的配置
- `activeTab`: 当前显示的图表标签

关键函数 & 行为:

- `initializeCharts()`: 按 DOM 初始化或销毁 ECharts 实例
- `fetchDataBySelection()`: 根据 `selectedDataSource` 与 `selectedDevice` 调用不同后端接口并填充 `accumulatedData`
- `preprocessChartOptions(data)`: 预生成所有标签页的 ECharts 配置并缓存到 `chartOptions`
- `generateAccelerometerOption` / `generateStrainGaugeOption`: 把原始数组转换为 ECharts option (包含标题、xAxis、yAxis、series、dataZoom 等)
- `drawAllCharts()` / `drawSpecificChart()`: 将配置应用到图表并触发 `resize`

调用的后端 API:

- `/data/get_second_data`, `/data/get_minute_data`, `/data/get_hourly_data`,
`/data/get_daily_data`, `/data/get_monthly_data`
 - 请求参数常见: `device_name`, `num`

数据格式:

- 后端返回: { x: [[ts, v], ...], y: ..., z: ... } 或 { ch1: [[ts,v], ...], ch2: ... }
- 前端内存结构:
 - `accumulatedData.x.times` = [ts,...]
 - `accumulatedData.x.values` = [v,...]

生命周期与性能要点:

- 在 `onMounted` 初始化图表并 fetch 设备列表
- 使用 `nextTick` 确保 DOM 可用再初始化 ECharts
- 在切换设备/数据源时清空 `chartOptions` 以防配置污染
- 在卸载时 `dispose` ECharts 释放内存

常见边界/异常情况:

- 后端返回空数组或字段缺失时需要降级 (有若干防御性判断在代码中)
- 大量秒级数据 (几十万点) 会导致浏览器内存与渲染卡顿, 应限制 `num` 或使用 `downsampling`

改进建议:

- 在数据量大时加入服务端 down-sampling 或前端采样 (LOD) 策略
- 将 `generate*Option` 逻辑拆分到独立的 util 文件, 便于单元测试
- 为 `chartOptions` 增加有效期或基于参数键的缓存以避免内存长期增长

2. 设备监控仪表盘 (`pages/vibration/dashboard.vue`)

概述与职责: `dashboard.vue` 是运维/监控人员每天查看的主控面板, 强调“实时性”和“可操作性”。它同时承载:

- 秒级实时监控 (默认轮询与刷新)
- 月级历史趋势展示 (用于趋势分析)
- 异常列表与筛选 (快速定位问题记录)
- 阈值查看与短期在线状态提示

模板结构:

- 顶部 header (页面标题与返回主页按钮)
- 左侧大图表区 (实时标签页 + 月级标签页)
- 右侧信息卡 (设备信息、阈值、在线状态)
- 右下角/底部异常记录列表 (带筛选、分页或导出口)

主要响应式状态:

- `selectedCategory, selectedDevice`: 设备筛选
- `currentDeviceInfo`: 根据 `selectedDevice` 推导的类型、标签等
- `currentRealtimeData, currentMonthlyData`: 从后端拉取的原始数组格式数据
- `realtimeCharts, monthlyCharts, chartOptions`: 图表实例与缓存配置
- `thresholds`: 阈值 (从后端 `/data/get_threshold_or_offset` 获取)

关键函数 & 行为:

- `initializeRealtimeCharts() / initializeMonthlyCharts()`: 初始化不同场景的图表实例
- `fetchRealtimeData()`: 调用 `/data/get_second_data`, 拉取秒级数据并触发预处理/绘图
- `fetchMonthlyData()`: 调用 `/data/get_monthly_data`, 用于趋势图
- `fetchAbnormalData()`: 调用 `/data/get_abnormal_data_with_type` 显示最近异常
- `preprocessChartOptions(data, isRealtime)`: 通用预处理函数, 生成带 `dataZoom` 的 option
- `startRealtimePolling()`: 开启定时器 (每5秒) 并安全清理旧定时器

调用的后端 API:

- `/data/get_second_data` (实时)
- `/data/get_monthly_data` (历史趋势)
- `/data/get_threshold_or_offset` (阈值、偏移)
- `/data/get_abnormal_data_with_type` (异常记录)

重要实现细节与防护:

- 在 `onDeviceChange()` 中会停止旧轮询、重置标签页并重新初始化图表, 防止并发请求污染 UI
- 对后端返回格式做了防御性检查 (存在性和数组长度), 避免空数据导致 ECharts 抛错
- 将实时/月级配置以 `prefix` (realtime/monthly) 区分缓存键, 便于复用

常见边界/异常情况:

- 后端长时间返回失败: `deviceStatus.online` 标记为离线并提示
- 秒级数据量非常大: 建议后端限制单次点数或在前端采用 window 滑动显示

改进建议:

- 将轮询频率与页面可见性绑定 (页面不可见时暂停轮询) 以节省资源
- 为异常列表添加实时 websocket 推送以替代高频轮询 (更低延迟和服务端负载)
- 为阈值展示与编辑增加权限控制 (防止误改)

3. 异常监控与统计 (`pages/vibration/abnormal.vue`)

概述与职责: 该页面用于对历史异常数据做精确查询、筛选与导出。典型用户场景: 运维需要按时间段、设备与方向 (x/y/z/ch) 导出数据、做统计或上报。页面职责包括:

- 提供级联选择器选择设备与方向 (自动根据设备类型过滤方向)
- 时间范围与快捷选项 (1周、1月、昨天等)
- 显示表格 (带分页/滚动) 并支持 CSV 导出

模板结构:

- 顶部: 设备级联 (`cascader`)、方向级联、时间范围选择器
- 中部: 查询/导出按钮
- 底部: 结果表格 (`el-table`) 或空状态提示

主要响应式状态:

- `params`: 查询参数 (device: array, direction: array, start_time, end_time)
- `chartData`: 表格数据数组
- `loading`: 加载状态
- `deviceList`、`cascaderDevices`: 用于构建级联选项的本地设备清单

关键函数 & 行为:

- `getDeviceId(deviceValue)`: 从级联值中解析实际设备 ID
- `getDirectionValue(directionValue)`: 解析选择的方向值
- `fetchData()`: 构造请求参数并调用 `/data/get_abnormal_data`, 对响应做兼容处理 (尝试补齐 device_id)
- `downloadData()`: 生成导出链接 `download_abnormal_data` 并通过 a 标签触发浏览器下载

调用的后端 API:

- `/data/get_abnormal_data`
- `/data/download_abnormal_data`

数据格式与兼容性策略:

- 期望后端返回项含 { device_id, device_name, time, data, direction, min, max }
- 前端在拿到数据后会尝试根据 device_name 反查 device_id, 保证导出和展示列完整

常见边界/异常情况:

- 级联组件返回数组路径，需谨慎取最后元素作为实际参数
- 时间范围选择要做前后判定（开始时间不可晚于结束时间），并限制可选区间
- 后端返回空或 error 要给用户明确提示（`ErrorMessage`）

改进建议:

- 对于大数据导出，后端提供异步导出任务（返回任务 id）会更稳健
- 将设备在线状态与结果行高亮结合，便于快速定位线下设备导致的数据缺失

4. 参数设置页面 (`pages/vibration/parameter.vue`)

概述与职责: `parameter.vue` 用来执行对监测设备的“配置下发”与“校准”，包括偏移（offset）和阈值（limit）的设置。典型场景：设备安装或重校准后，运维人员通过该页面将新参数写入后端数据库/设备。

模板结构:

- 设备选择网格（分建筑分组显示设备）
- 偏移输入、阈值输入（按设备类型显示不同字段 X/Y/Z 或 Ch1/Ch2）
- 应用按钮（批量提交多个 update 请求）
- 限值（邮箱/短信）设置与单独应用按钮

主要响应式状态:

- `selectedDevice`: 当前选中的设备对象（包含 value/label/type）
- `xOffset/yOffset/zOffset, xThreshold/...`: 当前输入的参数值
- `ratio_y_x` 等: 用于联动计算阈值的比例（从 `/data/get_ratio_data` 获取）

关键函数 & 行为:

- `updateSingle(item)`: 对单个参数调用 `/data/update_threshold_or_offset`，对响应做健壮解析（空响应或非 JSON 也当作成功处理）
- `applyCalibration()`: 按设备类型收集需要下发的参数并并行发送（`Promise.all`），统一显示结果
- `fetchThresholds()`: 获取当前设备阈值，填充表单
- `getRatioData()`: 获取轴间比例以便联动更新阈值输入

调用的后端 API:

- `/data/get_threshold_or_offset`（获取）
- `/data/update_threshold_or_offset`（更新，GET 接口风格）
- `/data/get_ratio_data`（获取比例）

安全性与幂等性注意:

- 当前实现使用 GET 请求携带修改参数（query string），这在 REST/幂等性方面不理想，建议改为 POST 且带 CSRF/权限校验
- 前端对 update 返回格式做了容错（解析失败仍视为成功），建议后端统一返回明确的 success 字段便于前端判断

常见边界/异常情况:

- 并发多次点“应用”可能引起重复请求，建议在按钮禁用期间加入防重校验
- 后端超时或返回异常 JSON 时，前端需告知用户并列出生失败项

改进建议:

- 把批量更新改为后端支持的单个批量接口，减少网络往返并保证事务一致性
- 增加变更历史记录（谁、何时、从什么值->到什么值），便于回滚与审计

API接口设计

数据获取接口

- GET /data/get_second_data - 秒级数据
- GET /data/get_minute_data - 分钟级数据
- GET /data/get_hourly_data - 小时级数据
- GET /data/get_daily_data - 天级数据
- GET /data/get_monthly_data - 月级数据

参数管理接口

- GET /data/get_threshold_or_offset - 获取阈值和偏移
- GET /data/update_threshold_or_offset - 更新阈值和偏移
- GET /data/get_ratio_data - 获取比例数据

异常数据接口

- GET /data/get_abnormal_data - 获取异常数据
- GET /data/download_abnormal_data - 导出异常数据

数据模型

传感器数据结构

```
// 加速度计数据
{
  x: [[timestamp, value], ...],
  y: [[timestamp, value], ...],
  z: [[timestamp, value], ...]
}

// 应变计数据
{
  ch1: [[timestamp, value], ...],
  ch2: [[timestamp, value], ...]
}
```

阈值配置

```
{
  // 加速度计阈值
  x_limit: number,
```

```
y_limit: number,  
z_limit: number,  
  
// 应变计阈值  
ch1_limit: number,  
ch2_limit: number,  
  
// 预警阈值  
email_limit: number,    // 邮件预警线  
message_limit: number  // 短信预警线  
}
```

异常检测机制

异常类型

1. **阈值超限**: 数据值超过设定的最大/最小阈值
2. **方向分类**: X/Y/Z轴或Ch1/Ch2通道的异常
3. **严重程度**: 警告(warning) vs 严重(alarm)

检测逻辑

```
const checkThreshold = (data, threshold) => {  
  return {  
    above_max: data > threshold.max,  
    below_min: data < threshold.min  
  }  
}
```